OPERATING SYSTEMS – ASSIGNMENT 1

TASK 6 – Evaluate the Cost of System Calls (4 Marks)
CO1–CO5 | Bloom's Level: K5 – Evaluate

Introduction

A system call allows a user program to request a kernel service. It costs more than an ordinary function call because it requires a controlled transition from user mode to kernel mode and validation by the operating system.

1. Why a system call costs more

An ordinary function call usually changes the program's execution location within the same process and privilege level. A system call transfers execution to the kernel, checks parameters and permissions, performs kernel work, and then returns to user mode. It may also involve scheduling or waiting for I/O. Therefore, frequent small system calls can reduce performance.

2. Effect of buffer size on read() and write() calls

When a file is copied using a buffer of b bytes, the approximate number of data read operations for a file of n bytes is ceil(n/b), with an additional read often needed to detect end-of-file. A small buffer causes many calls. A larger buffer reduces the number of calls, but uses more memory. For the assignment's 10,000-byte test file, the expected results are:
• Buffer 1 byte: 10,001 read() calls
• Buffer 512 bytes: 21 read() calls
• Buffer 4096 bytes: 4 read() calls.

3. How library buffering helps

Functions such as fread() and fwrite() can transfer data in blocks using an internal buffer. This reduces the number of transitions between user mode and kernel mode compared with reading or writing one byte at a time. Buffering improves efficiency, especially when an application processes many small pieces of a file.

4. Completed C program using read() and write()

The following program completes the assignment's TODO. It handles partial writes by continuing until all bytes read have been written.

C program

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
char buf[8192];
if (bufsize == 0 || bufsize > sizeof(buf))
return -1;

int in = open(src, O_RDONLY);
if (in < 0) return -1;

int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (out < 0) {
close(in);
return -1;
}

long calls = 0;
ssize_t n;

while (1) {
n = read(in, buf, bufsize);
calls++;

if (n == 0) break;
if (n < 0) {
if (errno == EINTR) continue;
calls = -1;
break;
}

ssize_t total = 0;
while (total < n) {
ssize_t w = write(out, buf + total, (size_t)(n - total));
```

```
if (w < 0) {
if (errno == EINTR) continue;
calls = -1;
break;
}
total += w;
}
if (calls < 0) break;
}

close(in);
close(out);
return calls;
}

int main(void)
{
FILE *f = fopen("records.dat", "w");
if (!f) return 1;

for (int i = 0; i < 10000; i++)
fputc('A' + i % 26, f);
fclose(f);

size_t sizes[] = {1, 512, 4096};
for (int i = 0; i < 3; i++)
printf("buffer %4zu bytes: %5ld read() calls\n",
sizes[i], copy_file("records.dat", "copy.dat", sizes[i]));

unlink("records.dat");
unlink("copy.dat");
return 0;
}
```

5. Compile and execute

Save the code as **copy.c**. Compile using:
gcc -std=c11 -Wall -Wextra copy.c -o copy
Run using:
./copy
To trace system calls, use:
strace -c ./copy

Expected program output from the assignment:
buffer 1 bytes: 10001 read() calls
buffer 512 bytes: 21 read() calls
buffer 4096 bytes: 4 read() calls

The exact strace summary may include additional reads made by program startup and library functions. Compare the read() count for the copy operation with the program's own counter.

6. Alternative concept: memory-mapped files (mmap)

With mmap(), a file is mapped into the process's virtual address space. The program accesses file contents through memory rather than repeatedly issuing read() and write() calls. This can reduce explicit system-call frequency after the mapping is established. The OS handles page loading and write-back. It may also reduce copying between kernel and user buffers. However, mapping and unmapping require system calls, page faults can occur during access, and very large files need careful virtual-memory and address-space management. The mapped region is not necessarily all resident in physical RAM at once.

Conclusion

A larger buffer reduces the number of read() calls for the assignment's file-copy workload. A 4096-byte buffer is more efficient than a 1-byte buffer in the expected test. The final buffer size should be selected using measurements on the actual hospital backup system.

Prepared from the corresponding task in the uploaded CSA04 Operating Systems Assignment 1 document. Add your name, register number, required screenshots and your own measured evidence wherever requested before submission.